

Patryk Popenda

Applied LLM / AI Engineer · Remote EU · based in Poland

patryk@ewwesolutions.work · +48 512 030 048 · ewwesolutions.work ·
github.com/ewwe72/ewwesolutions · linkedin.com/in/patryk-popenda

SUMMARY

Applied-LLM engineer shipping two production Polish-language SaaS products built on the Anthropic API + AWS Bedrock. Comfortable across the full LLM stack: prompt engineering, forced tool-use with strict JSON schemas, multi-modal (text + vision) routing, prompt caching, evaluation, schema-validated output, cost modelling, and security auditing of LLM output boundaries (prompt injection, delimiter spoofing, stored XSS). Solo founder by necessity — I own the model layer, the FastAPI service, the worker, the Postgres schema, the Stripe integration, and the prod VM. Background is high-volume Windows L2/L3 end-user remote support at a large industrial-automation org (767+ closed tickets across the full endpoint / identity / M365 surface), so I'm equally at home in Windows + PowerShell + Entra ID / Intune ops as I am in Linux + Docker. Looking for a remote-EU role where the LLM layer is genuinely load-bearing, not a wrapper.

SELECTED PRODUCTION WORK

Faktomat — Polish invoice IDP SaaS

2025 – 2026 · solo

Live at faktomat.ewwesolutions.work

Paying flow end-to-end: signup → email verification → Stripe top-up → PDF upload → LLM extraction → editable review → JPK_FA (Polish government) XML export. Charged at 0.50 PLN per processed invoice.

- **Bedrock Claude (Haiku) extraction pipeline with forced tool-use** — `tool_choice={"type": "tool", "name": ...}` + strict pydantic `TOOL_SCHEMA`, which eliminates the "model emits text outside the schema" failure mode entirely. Schema violations become server errors, not silent drops.
- **Multi-modal routing.** Text path uses pdfplumber/pypdf; if text extraction is degenerate, the same chunk is re-sent as a `document` block (rasterised pages) — vision-mode fallback at parity cost per the published Anthropic pricing.
- **Bounded blast-radius extraction.** Per-page cost ceiling, per-upload spend cap, friendly Polish error messages with raw LLM output preserved to `audit_events.payload.error_raw` for post-hoc debugging.
- **Stack:** FastAPI · pydantic-strict · SQLAlchemy + Alembic · arq worker · Postgres · Redis · MinIO (S3-compatible) · Stripe live (Checkout Sessions + webhooks) · Cloudflared tunnel · all containerised via docker-compose on a Hyper-V Ubuntu VM.
- **Discipline:** mypy `--strict` across the source tree, integration test suite that hits a real Postgres (no mocks at the DB boundary), pre-push secret-audit script, public `/status` page (DB / Redis / MinIO probes + 30 s cache + bounded boto3 timeouts), per-event Discord alerter with anchored cursor.

Fiszkomat — Polish PDF → Anki flashcard generator

2025 – 2026 · solo

Live at fiszkomat.ewwesolutions.work

Paid checkout, in-browser SM-style reviewer + `.apkg` export, 7 curated sample decks (296 cards) on the landing page.

- **Direct Anthropic API** (Claude Haiku 4.5) with **prompt caching** (`cache_control: ephemeral`) on the system prompt — meaningful per-run cost reduction since the system prompt is the chunked-call prefix.
- **Pydantic-validated card schema** (`(z, t, d, m, i, c, n)`) with sha1 dedup, length caps per field, and a Polish-only enforcement detector that inspects every visible card text field against an English-only stopword set chosen to avoid Polish false positives. Iterated from 5 hard-coded stopwords to a curated 47-word set after a static audit.

- **268-line static prompt-safety audit** I wrote against my own code (`docs/fiskomat-prompt-audit.md`) covering: delimiter-spoof injection via `---` chunk markers, Polish-only detector gaps, HTML-escape posture across three render surfaces (server-side sample HTML, in-browser reviewer DOM, Anki Mustache template), DoS surface paywalling, prompt caching as economic optimisation vs security control.
- **XSS regression tests as positive pins** on the LLM output path — if a future refactor swaps `html.escape` for raw interpolation or switches the reviewer from `textContent` to `innerHTML` , the test suite fails loudly with a comment pointing to the audit.
- **147 pytest tests** covering `validate_cards`, telemetry sanitisation (no `$X.XX` cost or token counts leak to `/status`), sample-deck asset integrity, and the audit findings.
- **Active dogfood user** giving structured feedback (a Polish medical student preparing for kolokwia).

Studio — ewwesolutions.work

2025 – 2026

Umbrella site for both products. Static HTML served by a tiny systemd unit; the equity card pulls live paper-trading P&L from a 30-minute Alpaca snapshot timer. Honest delta-from-baseline rendering — no cherry-picked highs.

CORE STACK

- **LLM:** Anthropic Claude (direct API + AWS Bedrock), prompt caching, forced tool-use with input schemas, vision/document blocks, structured output validation, multi-modal routing, cost modelling, prompt-injection threat-modelling, evaluation against real Polish-language inputs.
- **Backend:** Python 3.12, FastAPI, Pydantic (strict), SQLAlchemy + Alembic, asyncio, arq (Redis-backed task queue).
- **Data:** Postgres, Redis, MinIO (S3-compatible), boto3.
- **Infra (Linux):** Docker, docker-compose, Cloudflared, systemd, Ubuntu.
- **Infra (Windows):** Hyper-V, PowerShell (Exchange Online, MSGraph / AzureAD, Intune Graph API, ActiveDirectory module), Group Policy, Intune configuration profiles, Entra ID conditional access.
- **Payments:** Stripe (Checkout Sessions, live keys, webhook verification, idempotency).
- **Frontend:** Server-rendered Jinja2 + vanilla JS, minimal-dep philosophy, no SPA framework.
- **Discipline:** mypy `--strict` , pytest (unit + integration), pre-push secret audit, conventional commits, post-hoc rationale in CHANGELOG.

LANGUAGES

Polish English

native fluent — technical writing, async, on-call

PREVIOUS EXPERIENCE

Rockwell Automation — IT Specialist (L2 / L3), End-User Remote Support

2022 – 2024

End-of-line remote support for a large industrial-automation org. The service desk escalates to my queue; if I or my colleague can't close the ticket, it goes to ticket hell. Partner with field services for the physical-touch side. ServiceNow is the system of record.

- **767+ closed tickets across the full Windows-endpoint surface.** Wi-Fi 802.1x certificate failures on the corporate SSID, print queue / driver / printer-server issues, Windows 10/11 in-place upgrade failures, peripherals (wireless mouse, dock, monitor), line-of-business app installs (Visio, SAP GUI, vendor tools), RDP / RDS / SAP connectivity (*"the remote device or resource won't accept the connection"* and its cousins), VPN + conditional-access edge cases, OneDrive / profile corruption, BitLocker recovery, Autopilot enrolment. Whatever shape the ticket arrived in, the OS, network, identity, and app layer were all on me — no narrow lane.
- **Fast triage across a wide surface.** Reading Windows event logs, `gpreresult` , `dsregcmd` , certificate stores, MDM diagnostics, basic network captures — and knowing which layer to look at first based on the symptom. The part that doesn't show up on a CV but is what made the SLA: pattern-matching across hundreds of tickets so the second instance of a problem closes in minutes.

- **Direct end-user resolution over the queue handoff.** Defaulted to connecting straight to the affected user — remote session, call, or walk-up via field services — to close the ticket in one pass rather than ping-pong it through assignment groups. Faster SLA closure, fewer reopens, higher user-side trust.
- **PowerShell + Microsoft 365 admin** when scripting was faster than clicking. Exchange Online Management cmdlets, MSGraph / AzureAD for Entra ID, Intune Graph for device-config drift, ActiveDirectory module for on-prem AD. Recurring fixes were captured as runnable `.ps1` snippets pinned in the KB, not screenshot walkthroughs.
- **Change-approval gate for offshore ops.** Reviewed and signed off on production-VM restarts, maintenance windows, and infra changes executed by the outsourced ops vendor — the "I approve, proceed" reply that gates a prod-touching action.
- **Runbook + knowledge-base authoring.** Documented recurring fixes in the ServiceNow KB so the service desk could deflect the same ticket without another escalation to my queue.
- **Cross-timezone async written communication.** Most of the work was email and ticket comments to field services and to teams in IST and US-CT — the exact written-async muscle the LLM/SaaS work later leaned on.

EDUCATION

- **Self-taught software engineer.** Python, FastAPI, Anthropic API, AWS, Stripe, Docker, Postgres — all production-grade and shipping real revenue. Everything in *Selected production work* above was built outside any formal CS program.

LINKS

- Studio (umbrella): ewwesolutions.work
- Faktomat (live): faktomat.ewwesolutions.work
- Fiszkomat (live, sample decks browsable without payment): fiszkomat.ewwesolutions.work
- Code monorepo: github.com/ewwe72/ewwesolutions
- LinkedIn: linkedin.com/in/patryk-popenda